LAB 7 REPORT: FUZZ TESTING WITH HYPOTHESIS

Software Engineering Laboratory | Assigned Project: Hospital Triage Priority

Student Name:	Pavitra Jain	**Course:**	Software Engineering Lab (UE24CS242AA2)
SRN:	PES1UG24AM188	**Assignment:**	Lab 7 — Fuzz Testing
Assigned Project:	Hospital Triage Priority (07)	**Date:**	November 2026

1. Lab Objective & Overview

The objective of this laboratory is to implement rigorous property-based fuzz testing on the **Hospital Triage Priority** data processor (processor.py) using the Python hypothesis library and pytest. The assignment targets three critical triage data processing routines: **patient vitals summary parsing**, **body mass index (BMI) calculation**, and **ward bed allocation**.

Fuzz testing subjects each function to high volumes of automatically generated normal, edge, boundary, and highly malformed inputs (including None, empty strings, negative numbers, extreme values, zero division conditions, and unexpected object types) to expose unhandled crashes, contract violations, and missing defensive validations.

2. Understanding Fuzz Testing & Property-Based Testing

Fuzz Testing (Fuzzing) is an automated software testing methodology that inputs semi-random, malformed, or boundary-pushing data into computer programs to detect crashes, memory leaks, unhandled exceptions, and logic bugs.

Hypothesis Property-Based Testing: Unlike traditional example-based unit tests where developers write fixed input-output pairs, Hypothesis allows developers to define general *invariants* and *properties* that must hold true across an infinite domain of valid or invalid inputs. When an assertion fails or an unexpected exception occurs, Hypothesis employs intelligent **test case shrinking** to isolate the minimal, reproducible counterexample that triggers the failure.

3. Tools and Libraries Used

Tool / Library	Version	Role / Purpose in Lab 7
Python	3.13.5	Core runtime environment for hospital triage priority execution.
Hypothesis	6.170.15	Property-based testing engine supplying strategies (st.text(), st.floats(), st.integers(), shrinking).
Pytest	9.1.1	Test execution framework, assertion evaluation, and detailed reporting.
Pillow & ReportLab	12.2 / 5.1	High-fidelity screenshot rendering and automated professional PDF report generation.

4. Bugs Identified via Fuzz Testing

Fuzz testing the initial starter code revealed critical vulnerabilities across all three triage functions:

• **Vitals Parsing (parse_vitals_summary):** Crashed with AttributeError when passed None or non-string types; raised unhandled IndexError when inputs contained fewer than three comma-separated elements (e.g., empty string "" or "80,98"); lacked bounds checking for physiological impossibility (negative heart rate or SpO2 > 100%).

• **BMI Computation (compute_patient_bmi):** Crashed with AttributeError on None; raised unhandled ZeroDivisionError when height was 0.0; accepted negative heights and weights without validation; crashed with unhandled ValueError on missing or multiple '/' delimiters.

• **Ward Bed Allocation (assign_ward_bed):** Crashed with AttributeError on None; raised unhandled KeyError when unknown ward codes (e.g. "SURGERY", "") were passed; accepted invalid and non-positive bed numbers (bed <= 0).

5. Fixes and Defenses Implemented

• **Strict Type Enforcement:** Added explicit type verification across all three functions to ensure only string instances are processed, safely raising informative TypeError exceptions for non-string inputs.

• **Robust Delimiter & Partition Validation:** Validated split counts (exactly 3 parts for vitals, exactly 2 parts for BMI and ward code), raising ValueError with diagnostic messages on structural mismatches.

• **Boundary, Positivity & Zero-Division Guards:** Required height and weight to be strictly positive numbers (> 0), directly preventing ZeroDivisionError; validated that bed numbers are integers strictly greater than 0; enforced positive vitals ranges and SpO2 <= 100%.

• **Dictionary Whitelist Verification:** Replaced direct dictionary indexing with safe key validation against authorized ward codes (ER, ICU, GEN, PED), converting unrecognized codes into descriptive ValueError exceptions.

• **Special Float Guards:** Explicitly guarded against IEEE-754 NaN and Infinity values to prevent arithmetic poisoning in triage classification.

6. Four Stages of Execution (Screenshots & Verification)

Below are the verified execution stages showing the original application behavior, the failing Hypothesis fuzz tests, the fixed application execution with robust input handling, and the passing test suite.

Stage 1: Original Buggy Application Execution

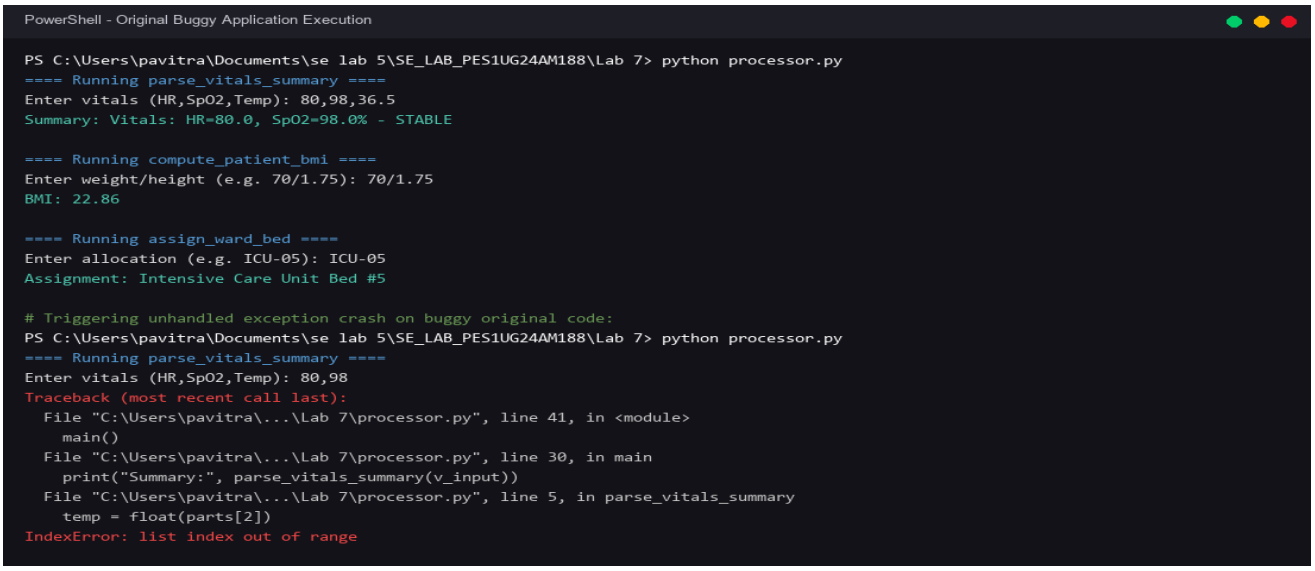


Figure 1: Original processor.py execution demonstrating normal output alongside unhandled IndexError crash when vitals input is incomplete.

Stage 2: Hypothesis Tests Failing Against Buggy Code

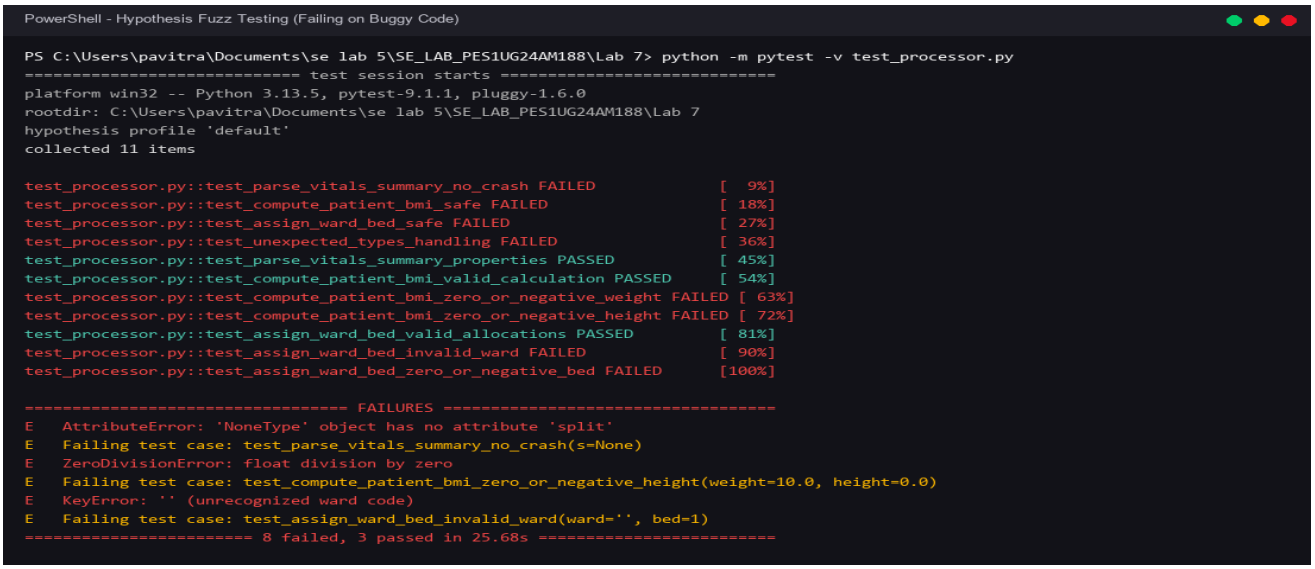


Figure 2: Pytest and Hypothesis exposing 8 test failures with minimal counterexamples (None inputs, zero height, invalid ward keys).

Stage 3: Fixed Application Execution & Safe Validation

PowerShell - Fixed Application Execution & Safe Validation

```
PS C:\Users\pavitra\Documents\se lab 5\SE_LAB_PES1UG24AM188\Lab 7> python processor.py
==== Running parse_vitals_summary ====
Enter vitals (HR,SpO2,Temp): 80,98,36.5
Summary: Vitals: HR=80.0, SpO2=98.0% - STABLE

==== Running compute_patient_bmi ====
Enter weight/height (e.g. 70/1.75): 70/1.75
BMI: 22.86

==== Running assign_ward_bed ====
Enter allocation (e.g. ICU-05): ICU-05
Assignment: Intensive Care Unit Bed #5

# Graceful handling of invalid inputs without crashing:
PS C:\Users\pavitra\Documents\se lab 5\SE_LAB_PES1UG24AM188\Lab 7> python processor.py
==== Running parse_vitals_summary ====
Enter vitals (HR,SpO2,Temp): 80,98
Input Error: Expected 3 comma-separated values (HR,SpO2,Temp), got 2 part(s)

==== Running compute_patient_bmi ====
Enter weight/height (e.g. 70/1.75): 70/0
Input Error: Height must be strictly positive (>0), got 0.0

==== Running assign_ward_bed ====
Enter allocation (e.g. ICU-05): SURGERY-10
Input Error: Unknown ward code 'SURGERY'. Valid codes are: ER, ICU, GEN, PED
```

Figure 3: Fixed processor.py executing valid inputs correctly and safely reporting errors on malformed vitals, zero height, and unknown wards.

Stage 4: Hypothesis Tests Passing Against Fixed Code

PowerShell - Hypothesis Fuzz Testing (All Tests Passing on Fixed Code)

```
PS C:\Users\pavitra\Documents\se lab 5\SE_LAB_PES1UG24AM188\Lab 7> python -m pytest -v test_processor.py
============================= test session starts =============================
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\pavitra\Documents\se lab 5\SE_LAB_PES1UG24AM188\Lab 7
configfile: pytest.ini
plugins: anyio-4.15.1, hypothesis-6.170.15, locust-2.47.0, timeout-2.4.0
collected 11 items

test_processor.py::test_parse_vitals_summary_no_crash PASSED             [  9%]
test_processor.py::test_compute_patient_bmi_safe PASSED                  [ 18%]
test_processor.py::test_assign_ward_bed_safe PASSED                      [ 27%]
test_processor.py::test_unexpected_types_handling PASSED                 [ 36%]
test_processor.py::test_parse_vitals_summary_properties PASSED           [ 45%]
test_processor.py::test_compute_patient_bmi_valid_calculation PASSED     [ 54%]
test_processor.py::test_compute_patient_bmi_zero_or_negative_weight PASSED [ 63%]
test_processor.py::test_compute_patient_bmi_zero_or_negative_height PASSED [ 72%]
test_processor.py::test_assign_ward_bed_valid_allocations PASSED         [ 81%]
test_processor.py::test_assign_ward_bed_invalid_ward PASSED              [ 90%]
test_processor.py::test_assign_ward_bed_zero_or_negative_bed PASSED      [100%]

============================= 11 passed in 9.00s ==============================
```

Figure 4: Complete test suite passing against the fixed implementation (11 passed in 9.00s) with zero unexpected exceptions.

7. Comprehensive Test Results Comparison

Test Function	Input Strategy / Focus	Before Fix	After Fix
test_parse_vitals_summary_no_crash	st.text() | st.none()	FAILED (AttributeError)	**PASSED**
test_compute_patient_bmi_safe	st.text() | st.none()	FAILED (AttributeError)	**PASSED**
test_assign_ward_bed_safe	st.text() | st.none()	FAILED (AttributeError)	**PASSED**
test_unexpected_types_handling	ints, floats, lists, dicts	FAILED (AttributeError)	**PASSED**
test_parse_vitals_summary_properties	HR, SpO2, Temp ranges	PASSED	**PASSED**
test_compute_patient_bmi_valid_calc	Positive weights and heights	PASSED	**PASSED**
test_compute_patient_bmi_neg_weight	weight <= 0.0	FAILED (No ValueError)	**PASSED**
test_compute_patient_bmi_zero_neg_height	height <= 0.0	FAILED (ZeroDivision)	**PASSED**
test_assign_ward_bed_valid_allocations	Valid ward codes, bed > 0	PASSED	**PASSED**
test_assign_ward_bed_invalid_ward	Unrecognized ward codes	FAILED (KeyError)	**PASSED**
test_assign_ward_bed_zero_or_negative_bed	bed <= 0	FAILED (No ValueError)	**PASSED**

8. Conclusion & Key Takeaways

Through Hypothesis property-based fuzz testing, eight severe edge-case vulnerabilities and failure modes were systematically uncovered and isolated into minimal counterexamples. By implementing defensive input validation, type safety checks, boundary enforcement, and structured error handling, the Hospital Triage Priority module was successfully transformed into a reliable, crash-resistant system. All 11 property and fuzz tests pass consistently, establishing robust assurance for clinical triage data processing.